NEXT ENGINEER · _SHEERAT_

Python Programming

BCC402

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 4 | **Credits:** 2

L-T-P: 2-0-0

Scheme: AKTU B.Tech 2nd year, 2023-24 onwards (same syllabus as BCC302 in semester 3)

UNIT 1 Introduction to Python

UNIT 2 Program Flow Control

UNIT 3 Complex Data Types and Functions

UNIT 4 File Operations

UNIT 5 Packages and GUI Programming

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: rules, tables and "what is the output" traps.
3. **Solved examples**: complete programs that are commonly asked, with their output.
4. **Code to remember**: short snippets to reproduce from memory.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (2 hours):** Units 1 and 2. Type or write every program once: you will be asked to write programs, and output-prediction questions are common.
- **Afternoon (2 to 3 hours):** Unit 3 is the biggest: strings, lists, tuples, dictionaries, functions. Learn the method tables.
- **Evening (2 hours):** Units 4 and 5 (files, numpy/pandas/matplotlib, Tkinter), then the "Last-hour checklist".
- **In the exam:** write programs with proper indentation, a comment at the top saying what the program does, and a sample output. Name the built-in method you use and what it returns.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to Python

✍️ 2-mark definitions

- **Python:** a high-level, interpreted, dynamically typed, general-purpose programming language that uses indentation to mark blocks.
- **Interpreted:** code is executed line by line by the Python interpreter (source → bytecode → Python Virtual Machine), with no separate compile step for the programmer.
- **Dynamic typing:** a variable's type is decided at run time by the value assigned to it, and can change: `x = 5` then `x = "hi"`.
- **Variable:** a name that refers to an object in memory. Python variables hold references, not the values themselves.
- **Identifier rules:** letters, digits and `_`; cannot start with a digit; case-sensitive; cannot be a keyword (`if`, `for`, `class`, ...).
- **Block / indentation:** a group of statements with the same indentation after a line ending in `:`. Python uses indentation (usually 4 spaces) instead of braces.
- **Comment:** text ignored by the interpreter: `# single line`; a triple-quoted string is often used for multi-line comments/docstrings.
- **Mutable vs immutable:** mutable objects can change in place (list, dict, set); immutable ones cannot (int, float, str, tuple).
- **`type()` and `id()`:** `type(x)` gives the class of `x`; `id(x)` gives its identity (memory address).

💡 Core concepts

Features of Python: simple and readable, free and open source, interpreted, portable, dynamically typed, object-oriented, huge standard library, extensible and embeddable.

Built-in data types

Category	Types	Example
Numeric	int, float, complex	`10`, `3.14`, `2+3j`
Boolean	bool	`True`, `False`
Sequence	str, list, tuple, range	`"abc"`, `[1, 2]`, `(1, 2)`, `range(5)`
Mapping	dict	`{"a": 1}`
Set	set, frozenset	`{1, 2, 3}`
None	NoneType	`None`

Operators

Type	Operators	Note
Arithmetic	`+ - * / // % **`	`/` always gives float; `//` floor division; `**` power

Type	Operators	Note
Comparison	`== != > < >= <=`	Return True/False
Logical	`and or not`	Short-circuit evaluation
Bitwise	`& | ^ ~ << >>`	Work on integer bits
Assignment	`= += -= *= /= //= %= **=`	
Identity	`is` , `is not`	Same object? (compares id)
Membership	`in` , `not in`	Element in a sequence?

Operator precedence (high to low): `()` → `**` → unary `+x -x ~x` → `* / // %` → `+ -` → shifts → `&` → `^` → `|` → comparisons, `is` , `in` → `not` → `and` → `or` . Note `**` is right-associative: `2 ** 3 ** 2 = 2 ** 9 = 512` .

Output traps examiners like

Expression	Result	Why
`7 / 2`	`3.5`	True division
`7 // 2`	`3`	Floor division
`-7 // 2`	`-4`	Floors toward −∞
`-7 % 3`	`2`	Result has the sign of the divisor
`2 ** 3 ** 2`	`512`	Right-associative
`0.1 + 0.2 == 0.3`	`False`	Floating-point rounding
`int("12") + 3`	`15`	Type conversion
`"5" * 3`	`"555"`	String repetition
`bool("")` , `bool([])` , `bool(0)`	`False`	Empty/zero values are falsy

Type conversion: implicit (int + float → float) and explicit: `int()` , `float()` , `str()` , `list()` , `tuple()` , `set()` .

Input/output: `input()` always returns a **string**, so convert: `n = int(input("Enter n: "))` . `print(a, b, sep=",", end="\n")` ; f-strings: `print(f"Sum = {a + b}")` .

🧮 Solved examples

Example 1 (swap two numbers without a third variable).

```
a, b = 5, 10
a, b = b, a          # tuple packing and unpacking
print(a, b)          # 10 5
```

Example 2 (area and perimeter of a circle).

```
import math
r = float(input("Radius: "))
print(f"Area = {math.pi * r ** 2:.2f}")
print(f"Perimeter = {2 * math.pi * r:.2f}")
# Radius: 7  →  Area = 153.94   Perimeter = 43.98
```

Example 3 (predict the output).

```
x = 10
y = x
y += 5
print(x, y)           # 10 15   (ints are immutable; y now refers to a new object)
a = [1, 2]
b = a
b.append(3)
print(a)              # [1, 2, 3]   (lists are mutable; a and b are the same object)
print(a is b)         # True
```

⌨️ Code to remember

```
print(type(5), type(5.0), type("5"), type([5]), type((5,)), type({5}), type({5: 1}))
# <class 'int'> <class 'float'> <class 'str'> <class 'list'> <class 'tuple'> <class 'set'> <class
'dict'>
n = int(input())                # read an integer
a, b = map(int, input().split()) # read two integers on one line
```

⭐ Important questions

1. Explain the features of Python and how a Python program is executed (interpreter, bytecode, PVM).
2. Explain the data types in Python with examples; distinguish mutable and immutable types.
3. Explain the operators in Python with examples and their precedence. *Include the `/` vs `//` and `is` vs `==` differences.*
4. What is indentation in Python? Why is it important? *Show an `IndentationError` example.*

UNIT 2 Program Flow Control

✍️ 2-mark definitions

- **Conditional statement:** runs a block only when a condition is True: `if` , `if…else` , `if…elif…else` , nested `if` .
- `for` **loop:** iterates over the items of any iterable (range, string, list, dictionary...).
- `while` **loop:** repeats a block while a condition stays True.
- `range(start, stop, step)` **:** produces integers from start up to (not including) stop: `range(1, 10, 2)` → 1, 3, 5, 7, 9.
- `break` **:** exits the nearest loop immediately.
- `continue` **:** skips the rest of the current iteration and goes to the next.
- `pass` **:** a null statement used as a placeholder where a statement is required.
- **Loop** `else` **:** a block after a loop that runs only if the loop finished **without** a `break` .

💡 Core concepts

Looping over different iterables

```
for i in range(5): ...                # 0 1 2 3 4
for ch in "abc": ...                  # a b c
for item in [10, 20, 30]: ...         # each element
for k in d: ...                       # dictionary keys
for k, v in d.items(): ...            # key-value pairs
for i, item in enumerate(lst): ...    # index and value
```

break vs continue vs pass

Statement	Effect
`break`	Leaves the loop completely
`continue`	Jumps to the next iteration
`pass`	Does nothing; keeps syntax valid

Python has no `do-while` **or** `switch` (since 3.10 there is `match-case`). Emulate do-while with `while True: … if cond: break` .

🧮 Solved examples

Example 1 (prime check, using loop-else).

```
n = int(input("n: "))
if n < 2:
    print("Not prime")
else:
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            print("Not prime")
            break
    else:
        print("Prime")       # runs only if no break happened
```

Example 2 (factorial and Fibonacci).

```
n = 5
fact = 1
for i in range(1, n + 1):
    fact *= i
print(fact)                  # 120

a, b = 0, 1
for _ in range(7):
    print(a, end=" ")        # 0 1 1 2 3 5 8
    a, b = b, a + b
```

Example 3 (reverse a number, palindrome and Armstrong check).

```
n = int(input())
temp, rev, arm, digits = n, 0, 0, len(str(n))
while temp > 0:
    d = temp % 10
    rev = rev * 10 + d
    arm += d ** digits
    temp //= 10
print("Palindrome" if rev == n else "Not palindrome")
print("Armstrong" if arm == n else "Not Armstrong")     # 153 → Armstrong
```

Example 4 (pattern).

```
for i in range(1, 5):
    print("*" * i)
# *
# **
# ***
# ****
```

Example 5 (predict the output).

```
for i in range(1, 10):
    if i % 2 == 0:
        continue
    if i > 7:
        break
    print(i, end=" ")         # 1 3 5 7
```

⌨️ Code to remember

```
# largest of three
a, b, c = 3, 9, 5
if a >= b and a >= c: big = a
elif b >= c:          big = b
else:                 big = c

# sum of digits
s = sum(int(d) for d in str(1234))      # 10

# GCD
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

⭐ Important questions

1. Explain conditional statements in Python (if, if-else, elif, nested) with a program. *E.g. grade calculator or largest of three.*
2. Explain `for` and `while` loops with examples; write a program for prime numbers / Fibonacci / factorial.
3. Explain `break`, `continue`, `pass` and the `else` clause of loops with examples.
4. Write programs for Armstrong number, palindrome, sum of digits, and star/number patterns. *Show output.*

UNIT 3 Complex Data Types and Functions

✍️ 2-mark definitions

- **String:** an immutable sequence of Unicode characters, in single, double or triple quotes.
- **List:** an ordered, mutable collection that can hold mixed types: `[1, "a", 2.5]`.
- **Tuple:** an ordered, immutable collection: `(1, 2, 3)`. A one-element tuple needs a comma: `(5,)`.
- **Dictionary:** an unordered (insertion-ordered since 3.7), mutable mapping of unique, immutable keys to values: `{"name": "Ravi", "age": 20}`.
- **Set:** an unordered collection of unique elements: `{1, 2, 3}`.
- **Slicing:** extracting part of a sequence with `seq[start:stop:step]`.
- **Function:** a named, reusable block of code defined with `def` that may take parameters and return a value.
- **Lambda function:** a small anonymous function written in one expression: `square = lambda x: x * x`.
- **Recursion:** a function calling itself, with a base case.
- **Scope (LEGB):** Python looks up names in Local → Enclosing → Global → Built-in order. `global` lets a function change a global variable.

💡 Core concepts

Indexing and slicing (s = "PYTHON")

Expression	Result
`s[0]`, `s[-1]`	`'P'`, `'N'`
`s[1:4]`	`'YTH'`
`s[:3]`, `s[3:]`	`'PYT'`, `'HON'`
`s[::2]`	`'PTO'`
`s[::-1]`	`'NOHTYP'` (reverse)

String methods

Method	Does
`upper()`, `lower()`, `title()`, `capitalize()`, `swapcase()`	Change case
`strip()`, `lstrip()`, `rstrip()`	Remove spaces
`find(x)` / `index(x)`	Position of x (`find` gives −1, `index` raises error if missing)
`count(x)`	Number of occurrences

Method	Does
`replace(a, b)`	Replace a with b
`split(sep)` / `sep.join(list)`	String ↔ list
`startswith()`, `endswith()`	Prefix/suffix test
`isalpha()`, `isdigit()`, `isalnum()`, `isspace()`, `isupper()`	Character tests

List methods

Method	Does
`append(x)`	Add x at the end
`extend(iter)`	Add every item of iter
`insert(i, x)`	Insert x at position i
`remove(x)`	Remove first x (error if absent)
`pop(i)`	Remove and return item at i (last by default)
`sort()`, `sort(reverse=True)`	Sort in place; `sorted(lst)` returns a new list
`reverse()`	Reverse in place
`index(x)`, `count(x)`	Position, occurrences
`copy()`, `clear()`	Shallow copy, empty the list

Dictionary methods: `d[key]` (error if missing), `d.get(key, default)`, `d.keys()`, `d.values()`, `d.items()`, `d.update(other)`, `d.pop(key)`, `d.popitem()`, `d.setdefault(k, v)`, `del d[key]`.

List vs tuple vs set vs dict

	List	Tuple	Set	Dict
Brackets	`[ ]`	`( )`	`{ }`	`{k: v}`
Ordered	Yes	Yes	No	Yes (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	No	Keys unique
Indexing	Yes	Yes	No	By key

Functions

- **Arguments:** positional, keyword (`f(b=2, a=1)`), default (`def f(a, b=10)`), variable-length `*args` (tuple) and `**kwargs` (dict).
- A function without `return` returns `None`. A function can return several values as a tuple: `return a, b`.
- **Mutable default argument trap:** `def f(x, lst=[])` shares the same list between calls; use `lst=None` instead.

- **Built-ins with functions:** `map(f, iter)`, `filter(f, iter)`, `sorted(iter, key=f)`, `zip(a, b)`, `functools.reduce(f, iter)`.
- **List comprehension:** `[x * x for x in range(10) if x % 2 == 0]` → `[0, 4, 16, 36, 64]`.

🧮 Solved examples

Example 1 (count vowels, words and reverse words).

```
s = "Python is easy to learn"
vowels = sum(1 for ch in s.lower() if ch in "aeiou")
print(vowels)                          # 7
print(len(s.split()))                  # 5
print(" ".join(s.split()[::-1]))       # learn to easy is Python
```

Example 2 (frequency of characters with a dictionary).

```
s = "banana"
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)                            # {'b': 1, 'a': 3, 'n': 2}
```

Example 3 (list operations: second largest, remove duplicates).

```
nums = [4, 7, 2, 7, 9, 4]
unique = sorted(set(nums))             # [2, 4, 7, 9]
print(unique[-2])                      # 7  (second largest)
evens = [x for x in nums if x % 2 == 0]
print(evens)                           # [4, 2, 4]
```

Example 4 (recursive function).

```
def fact(n):
    return 1 if n <= 1 else n * fact(n - 1)
print(fact(5))                         # 120
```

Example 5 (arguments).

```
def info(name, age=18, *marks, **extra):
    print(name, age, marks, extra)
info("Asha", 20, 85, 90, city="Lucknow")
# Asha 20 (85, 90) {'city': 'Lucknow'}
```

Example 6 (predict the output).

```
t = (1, 2, [3, 4])
t[2].append(5)
print(t)                    # (1, 2, [3, 4, 5])  the tuple is immutable, the list inside is
not
print(list(map(lambda x: x * 2, [1, 2, 3])))      # [2, 4, 6]
print(list(filter(lambda x: x > 1, [0, 1, 2, 3]))) # [2, 3]
```

⌨️ Code to remember

```
s == s[::-1]                              # palindrome string check
d = dict(zip(["a", "b"], [1, 2]))         # {'a': 1, 'b': 2}
squares = {x: x * x for x in range(1, 4)}  # {1: 1, 2: 4, 3: 9}
max(d, key=d.get)                         # key with the largest value
sorted(words, key=len)                    # sort strings by length
```

⭐ Important questions

1. Explain string slicing and string methods with examples; write programs on strings (palindrome, vowel count, word reversal).
2. Differentiate list, tuple, set and dictionary. *Table + example of each.*
3. Explain list methods and list comprehension with examples.
4. Explain dictionary operations; write a program to count word/character frequency.
5. Explain functions: types of arguments, return values, lambda, recursion and scope. *Program for each.*
6. Explain `map`, `filter` and `reduce` with examples.

UNIT 4 File Operations

✍️ 2-mark definitions

- **File:** a named location on disk that stores data permanently. Python handles **text files** and **binary files**.
- `open(filename, mode)` **:** opens a file and returns a file object.
- **File modes:** `'r'` read (default, error if missing), `'w'` write (creates or **overwrites**), `'a'` append (creates or adds at end), `'x'` exclusive create (error if exists), `'r+'` read and write, `'b'` binary (e.g. `'rb'`), `'t'` text (default).
- `close()` **:** releases the file; unsaved buffered data is written.
- `with` **statement:** opens a file and closes it automatically, even if an error occurs.
- **File pointer:** the current position for the next read or write. `tell()` returns it; `seek(offset, whence)` moves it (whence 0 = start, 1 = current, 2 = end).

💡 Core concepts

Reading and writing functions

Function	Returns / does
`read()`	Whole file as one string
`read(n)`	Next n characters
`readline()`	Next line (including `\n`)
`readlines()`	List of all lines
`write(s)`	Writes string s; returns number of characters
`writelines(list)`	Writes each string in the list (adds no newlines)
`for line in f:`	Reads line by line (memory-efficient)

`seek` **in text vs binary mode:** in text mode only `seek(0)` from start (or `seek(0, 2)` to the end) is reliable; relative seeks need binary mode.

Exceptions with files: wrap risky code in `try / except FileNotFoundError / finally`.

`os` **module helpers:** `os.path.exists()`, `os.remove()`, `os.rename()`, `os.listdir()`, `os.getcwd()`.

🧮 Solved examples

Example 1 (write, then read back).

```
with open("data.txt", "w") as f:
    f.write("Hello\n")
    f.writelines(["Python\n", "AKTU\n"])

with open("data.txt") as f:
    print(f.readline(), end="")       # Hello
    print(f.readlines())              # ['Python\n', 'AKTU\n']
```

Example 2 (count lines, words and characters).

```
lines = words = chars = 0
with open("data.txt") as f:
    for line in f:
        lines += 1
        words += len(line.split())
        chars += len(line)
print(lines, words, chars)            # 3 3 18
```

Example 3 (copy one file to another, skipping lines that start with '#').

```
with open("src.txt") as src, open("dst.txt", "w") as dst:
    for line in src:
        if not line.startswith("#"):
            dst.write(line)
```

Example 4 (seek and tell).

```
with open("data.txt", "rb") as f:
    f.seek(6)             # move to byte 6
    print(f.read(6))      # b'Python'
    print(f.tell())       # 12
    f.seek(-5, 2)         # 5 bytes before the end
    print(f.read())       # b'AKTU\n'
```

⌨️ Code to remember

```
try:
    f = open("marks.txt")
    data = f.read()
except FileNotFoundError:
    print("File not found")
finally:
    print("Done")
# Append a record
with open("log.txt", "a") as f:
    f.write("new entry\n")
```

⭐ Important questions

1. Explain file handling in Python: opening modes, reading and writing functions, and closing a file. *Table of modes + program.*
2. Differentiate `read()`, `readline()` and `readlines()`; `write()` and `writelines()`. *With output.*
3. Explain `seek()` and `tell()` with an example.
4. Write a program to count lines, words and characters in a file / copy contents of one file to another / find the longest word in a file.
5. Why use the `with` statement? Explain exception handling with files.

NEXT ENGINEER
sheerat

UNIT 5 Packages and GUI Programming

✍️ 2-mark definitions

- **Module:** a single `.py` file containing functions, classes and variables, used with `import`.
- **Package:** a directory of modules (traditionally with `__init__.py`), e.g. `numpy`, `pandas`.
- **pip:** Python's package installer: `pip install numpy`.
- **NumPy:** a package for fast numerical computing with n-dimensional arrays (`ndarray`).
- **Pandas:** a package for data analysis with `Series` (1-D labelled) and `DataFrame` (2-D table) objects.
- **Matplotlib:** a plotting package; `matplotlib.pyplot` draws line, bar, pie, scatter and histogram charts.
- **Tkinter:** Python's standard GUI library, built on the Tk toolkit.
- **Widget:** a GUI element such as a button, label or text box.
- **Event loop (`mainloop()`):** keeps the window open and responds to user events.
- **IDE:** an integrated development environment (IDLE, PyCharm, VS Code, Jupyter, Spyder) with editor, run and debug tools.

💡 Core concepts

Import styles: `import math`, `import numpy as np`, `from math import sqrt, pi`, `from math import *` (not recommended).

NumPy essentials

```
import numpy as np
a = np.array([1, 2, 3])
b = np.arange(0, 10, 2)        # [0 2 4 6 8]
z = np.zeros((2, 3)); o = np.ones(3); l = np.linspace(0, 1, 5)
a * 2                          # [2 4 6]   vectorized, no loop
a.shape, a.ndim, a.dtype, a.sum(), a.mean(), a.max()
m = np.array([[1, 2], [3, 4]])
m.T, m @ m, np.dot(m, m)       # transpose, matrix product
```

NumPy arrays are faster and use less memory than lists because they store one data type contiguously and run operations in compiled code.

Pandas essentials

```
import pandas as pd
df = pd.DataFrame({"name": ["A", "B", "C"], "marks": [78, 92, 65]})
df.head(); df.info(); df.describe()
df["marks"].mean()                    # 78.33
df[df["marks"] > 70]                  # filter rows
df.sort_values("marks", ascending=False)
df = pd.read_csv("students.csv"); df.to_csv("out.csv", index=False)
```

Matplotlib essentials

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]; y = [10, 20, 15, 25]
plt.plot(x, y, marker="o", label="Sales")   # line chart
plt.bar(x, y); plt.pie(y, labels=x); plt.scatter(x, y); plt.hist(y)
plt.xlabel("Month"); plt.ylabel("Units"); plt.title("Report"); plt.legend()
plt.show()
```

Tkinter widgets

Widget	Use
`Label`	Display text or image
`Button`	Clickable button with `command=`
`Entry`	Single-line text input
`Text`	Multi-line text
`Checkbutton`, `Radiobutton`	Options
`Listbox`, `Scrollbar`	Lists
`Frame`	Container to group widgets
`Canvas`	Drawing shapes
`messagebox`	Pop-up dialogs

Geometry managers: `pack()` (stack in order), `grid(row, column)` (table), `place(x, y)` (exact position).

🧮 Solved examples

Example 1 (Tkinter: add two numbers).

```
import tkinter as tk

def add():
    result.config(text=str(int(e1.get()) + int(e2.get())))

root = tk.Tk()
root.title("Adder")
e1 = tk.Entry(root); e1.grid(row=0, column=0)
e2 = tk.Entry(root); e2.grid(row=0, column=1)
tk.Button(root, text="Add", command=add).grid(row=1, column=0)
result = tk.Label(root, text="0"); result.grid(row=1, column=1)
root.mainloop()
```

Example 2 (NumPy statistics).

```
import numpy as np
marks = np.array([56, 78, 90, 45, 67])
print(marks.mean(), np.median(marks), marks.std().round(2))   # 67.2 67.0 15.84
print(marks[marks > 60])                                      # [78 90 67]
```

Example 3 (Pandas + Matplotlib bar chart).

```
import pandas as pd, matplotlib.pyplot as plt
df = pd.DataFrame({"subject": ["DS", "COA", "DSTL"], "marks": [82, 75, 88]})
plt.bar(df["subject"], df["marks"], color="teal")
plt.title("Semester 3 marks"); plt.ylabel("Marks")
plt.show()
```

⌨️ Code to remember

```
from tkinter import messagebox
messagebox.showinfo("Title", "Saved!")
np.array([[1, 2], [3, 4]]).reshape(4)      # [1 2 3 4]
df.groupby("branch")["marks"].mean()       # average per group
```

⭐ Important questions

1. What are modules and packages? Explain different ways of importing, with examples.
2. Explain NumPy arrays and their advantages over lists; write programs using NumPy functions.
3. Explain Pandas Series and DataFrame with examples (create, read CSV, filter, sort, describe).
4. Explain plotting with Matplotlib (line, bar, pie charts) with a program.
5. What is Tkinter? Explain its widgets and geometry managers; write a GUI program (calculator / login form / adder).

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ Features of Python; interpreter → bytecode → PVM.
- ☐ Data types; mutable vs immutable.
- ☐ `/` vs `//`, `%` with negatives, `**` right-associative, `is` vs `==`.
- ☐ if/elif/else; for with range; while; break, continue, pass, loop-else.
- ☐ Prime, factorial, Fibonacci, palindrome, Armstrong programs.
- ☐ Slicing `s[::-1]`; string, list, dict method tables.
- ☐ List vs tuple vs set vs dict table.
- ☐ Function arguments: default, keyword, `*args`, `**kwargs`; lambda, map, filter.
- ☐ File modes r/w/a/x; read/readline/readlines; write/writelines; seek/tell; `with`.
- ☐ NumPy array basics; Pandas DataFrame basics; Matplotlib plot/bar/pie.

- [ ] Tkinter: Tk(), widgets, pack/grid/place, mainloop().

Books prescribed by AKTU

These are the books named in the AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Core Python Applications Programming (3rd ed.)	Wesley J. Chun	Pearson Education	WorldCat
2	Fundamentals of Python: First Programs (2nd ed.)	Kenneth A. Lambert	Cengage Learning	WorldCat
3	Introduction to Computer Science Using Python	Charles Dierbach	Wiley	WorldCat
4	Introduction to Computing and Problem Solving with Python	Jeeva Jose, P. Sojan Lal	Khanna Publishers	WorldCat
5	How to Think Like a Computer Scientist: Learning with Python	Allen B. Downey et al.	John Wiley	WorldCat
6	Learning Python (5th ed.)	Mark Lutz	O'Reilly	WorldCat
7	Python Programming: An Introduction to Computer Science (2nd ed.)	John Zelle	Franklin, Beedle / Cengage	WorldCat
8	Python Programming for the Absolute Beginner (3rd ed.)	Michael Dawson	Course Technology Cengage	WorldCat
9	Python Cookbook (3rd ed.)	David Beazley, Brian K. Jones	O'Reilly	WorldCat

The official Python tutorial at docs.python.org is free and legal, and covers Units 1 to 4 well.

Official syllabus: AKTU B.Tech 2nd year CSE syllabus, 2023-24 scheme (PDF)